JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY ANANTAPUR

DATA STRUCTURES LAB

Course Code: 23A05201P | R23 Regulations

Programs in C with Source Code and Output

Programme	B.Tech (AI & ML) - II Semester
Course	Data Structures Lab
L - T - P - C	0 - 0 - 3 - 1.5
Language / Compiler	C (ISO C11), gcc 13.3.0, flags: -Wall -Wextra -std=c11
Programs	24 programs in 9 exercises (all compiled with zero warnings)

Course Objectives

- To implement basic data structures: arrays, linked lists, stacks and queues.
- To implement searching, sorting and tree/hash based techniques.
- To apply data structures to solve real problems such as expression evaluation, parentheses matching and caching.

Course Outcomes

- CO1: Implement linear data structures (arrays, linked lists) and their operations.
- CO2: Implement searching and sorting techniques.
- CO3: Implement stacks and queues and apply them to expressions.
- CO4: Implement binary search trees and traversals.
- CO5: Implement hashing techniques and cache mechanisms.

Index of Programs

Ex.	Program
1(i)	Reverse an array
1(ii)	Searching techniques - Linear and Binary search
1(iii)	Sorting techniques - Bubble, Selection and Insertion sort
2(i)	Singly linked list - insertion and deletion
2(ii)	Reverse a linked list - iteratively and recursively
2(iii)	Problems on linked list traversal and manipulation
3(i)	Detect and remove duplicates from a linked list
3(ii)	Polynomial representation and addition using linked lists
3(iii)	Double-ended queue (deque) with essential operations
4(i)	Stack ADT using arrays and linked lists
4(ii)	Infix to postfix conversion and postfix evaluation
4(iii)	Parentheses matching and palindrome check using a stack
5(i)	Linear queue using arrays and linked lists
5(ii)	Circular queue using arrays
5(iii)	Palindrome check using a queue and a stack; priority queue
6(i)	Binary Search Tree - create, insert and search
6(ii)	BST - deletion, minimum, maximum, height and count
6(iii)	Tree traversals - inorder, preorder, postorder and level order
7(i)	Hash table with separate chaining
7(ii)	Hash table with open addressing - linear probing
8(i)	Hash table - quadratic probing and double hashing
8(ii)	Hashing applications - word frequency count and two-sum
9(i)	LRU cache using a hash map and doubly linked list
9(ii)	Cache simulation - direct mapped cache with hit ratio

Exercise 1: Arrays: Reversal, Searching and Sorting

Program 1(i) Reverse an array

Aim

To write a C program to reverse the elements of an array.

Algorithm

1. Take two indices: i at the first element and j at the last element.
2. Swap a[i] and a[j]; then do i++ and j--.
3. Repeat while i < j. The array is now reversed in place (n/2 swaps).

Program

```c
#include <stdio.h>

void display(const char *msg, int a[], int n)
{
    printf("%s: ", msg);
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void reverse(int a[], int n)
{
    int i = 0, j = n - 1;
    while (i < j) {
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;
        i++;
        j--;
    }
}

int main(void)
{
    int a[] = {10, 20, 30, 40, 50, 60, 70};
    int n = sizeof(a) / sizeof(a[0]);

    display("Original array", a, n);
    reverse(a, n);
    display("Reversed array", a, n);
    return 0;
}
```

Output

```
Original array: 10 20 30 40 50 60 70
Reversed array: 70 60 50 40 30 20 10
```

Result: The program was executed successfully and the output was verified.

Program 1(ii) Searching techniques - Linear and Binary search

Aim

To write C programs to implement Linear Search and Binary Search.

Algorithm

1. Linear search: compare the key with every element from index 0 until found or the array ends. Time O(n).
2. Binary search (array must be sorted): find mid = (low + high) / 2.
3. If a[mid] == key, stop; if key < a[mid] search the left half (high = mid - 1); else search the right half (low = mid + 1).
4. Repeat while low <= high. Time O(log n).

Program

```c
#include <stdio.h>

int linearSearch(int a[], int n, int key)
{
    for (int i = 0; i < n; i++)
        if (a[i] == key)
            return i;
    return -1;
}

int binarySearch(int a[], int n, int key)
{
    int low = 0, high = n - 1, step = 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        printf("  step %d: low=%d high=%d mid=%d a[mid]=%d\n",
               step++, low, high, mid, a[mid]);
        if (a[mid] == key)
            return mid;
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}

int main(void)
{
    int a[] = {12, 5, 9, 31, 24, 7, 18};
    int s[] = {5, 12, 18, 23, 31, 40, 56, 72};
    int n = sizeof(a) / sizeof(a[0]);
    int m = sizeof(s) / sizeof(s[0]);
    int keys1[] = {24, 100};
    int keys2[] = {31, 50};
    int pos;

    printf("Linear search in {12 5 9 31 24 7 18}\n");
    for (int i = 0; i < 2; i++) {
        pos = linearSearch(a, n, keys1[i]);
        if (pos >= 0) printf("  %d found at index %d\n", keys1[i], pos);
        else          printf("  %d not found\n", keys1[i]);
    }

    printf("\nBinary search in {5 12 18 23 31 40 56 72}\n");
```

```
    for (int i = 0; i < 2; i++) {
        printf("Searching %d\n", keys2[i]);
        pos = binarySearch(s, m, keys2[i]);
        if (pos >= 0) printf("  %d found at index %d\n", keys2[i], pos);
        else          printf("  %d not found\n", keys2[i]);
    }
    return 0;
}
```

Output

```
Linear search in {12 5 9 31 24 7 18}
  24 found at index 4
  100 not found

Binary search in {5 12 18 23 31 40 56 72}
Searching 31
  step 1: low=0 high=7 mid=3 a[mid]=23
  step 2: low=4 high=7 mid=5 a[mid]=40
  step 3: low=4 high=4 mid=4 a[mid]=31
  31 found at index 4
Searching 50
  step 1: low=0 high=7 mid=3 a[mid]=23
  step 2: low=4 high=7 mid=5 a[mid]=40
  step 3: low=6 high=7 mid=6 a[mid]=56
  50 not found
```

Result: The program was executed successfully and the output was verified.

Program 1(iii) Sorting techniques - Bubble, Selection and Insertion sort

Aim

To write C programs to implement Bubble sort, Selection sort and Insertion sort.

Algorithm

1. Bubble sort: repeatedly compare adjacent elements and swap if out of order; the largest element "bubbles" to the end in each pass.
2. Selection sort: in each pass find the minimum of the unsorted part and swap it to the front of that part.
3. Insertion sort: take the next element and insert it at its correct place in the already sorted left part by shifting larger elements right.
4. All three have O(n^2) time in the worst case.

Program

```
#include <stdio.h>

void printArray(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%3d", a[i]);
    printf("\n");
}

void bubbleSort(int a[], int n)
```

```c
{
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        }
        printf("  Pass %d:", i + 1);
        printArray(a, n);
        if (!swapped) break;
    }
}

void selectionSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min])
                min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
        printf("  Pass %d:", i + 1);
        printArray(a, n);
    }
}

void insertionSort(int a[], int n)
{
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
        printf("  Pass %d:", i);
        printArray(a, n);
    }
}

int main(void)
{
    int orig[] = {64, 25, 12, 22, 11, 90, 3};
    int n = sizeof(orig) / sizeof(orig[0]);
    int a[7];

    printf("Original array:");
    printArray(orig, n);

    for (int i = 0; i < n; i++) a[i] = orig[i];
    printf("\nBubble sort\n");
    bubbleSort(a, n);

    for (int i = 0; i < n; i++) a[i] = orig[i];
    printf("\nSelection sort\n");
    selectionSort(a, n);

    for (int i = 0; i < n; i++) a[i] = orig[i];
    printf("\nInsertion sort\n");
    insertionSort(a, n);

    printf("\nSorted array:");
    printArray(a, n);
```

```
    return 0;
}
```

Output

```
Original array: 64 25 12 22 11 90  3

Bubble sort
  Pass 1: 25 12 22 11 64  3 90
  Pass 2: 12 22 11 25  3 64 90
  Pass 3: 12 11 22  3 25 64 90
  Pass 4: 11 12  3 22 25 64 90
  Pass 5: 11  3 12 22 25 64 90
  Pass 6:  3 11 12 22 25 64 90

Selection sort
  Pass 1:  3 25 12 22 11 90 64
  Pass 2:  3 11 12 22 25 90 64
  Pass 3:  3 11 12 22 25 90 64
  Pass 4:  3 11 12 22 25 90 64
  Pass 5:  3 11 12 22 25 90 64
  Pass 6:  3 11 12 22 25 64 90

Insertion sort
  Pass 1: 25 64 12 22 11 90  3
  Pass 2: 12 25 64 22 11 90  3
  Pass 3: 12 22 25 64 11 90  3
  Pass 4: 11 12 22 25 64 90  3
  Pass 5: 11 12 22 25 64 90  3
  Pass 6:  3 11 12 22 25 64 90

Sorted array:  3 11 12 22 25 64 90
```

Result: The program was executed successfully and the output was verified.

Exercise 2: Singly Linked Lists

Program 2(i) Singly linked list - insertion and deletion

Aim

To implement a singly linked list and perform insertion and deletion operations.

Algorithm

1. Each node stores data and a pointer next; head points to the first node.
2. Insert at beginning: new->next = head; head = new. Insert at end: walk to the last node and link the new node. Insert at position: stop at node pos-1 and link in.
3. Delete from beginning: head = head->next. Delete from end: stop at the second last node. Delete by value: keep a previous pointer and bypass the node.
4. Free every deleted node to avoid memory leaks.

Program

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *createNode(int value)
{
    struct node *n = malloc(sizeof(struct node));
    n->data = value;
    n->next = NULL;
    return n;
}

struct node *insertBegin(struct node *head, int value)
{
    struct node *n = createNode(value);
    n->next = head;
    return n;
}

struct node *insertEnd(struct node *head, int value)
{
    struct node *n = createNode(value), *t = head;
    if (head == NULL) return n;
    while (t->next != NULL) t = t->next;
    t->next = n;
    return head;
}

struct node *insertPos(struct node *head, int value, int pos)   /* 1-based */
{
    if (pos == 1) return insertBegin(head, value);
    struct node *t = head;
    for (int i = 1; i < pos - 1 && t != NULL; i++) t = t->next;
    if (t == NULL) { printf("Invalid position %d\n", pos); return head; }
    struct node *n = createNode(value);
    n->next = t->next;
    t->next = n;
    return head;
```

```c
}

struct node *deleteBegin(struct node *head)
{
    if (head == NULL) { printf("List is empty\n"); return NULL; }
    struct node *t = head;
    head = head->next;
    printf("Deleted %d from beginning\n", t->data);
    free(t);
    return head;
}

struct node *deleteEnd(struct node *head)
{
    if (head == NULL) { printf("List is empty\n"); return NULL; }
    if (head->next == NULL) {
        printf("Deleted %d from end\n", head->data);
        free(head);
        return NULL;
    }
    struct node *t = head;
    while (t->next->next != NULL) t = t->next;
    printf("Deleted %d from end\n", t->next->data);
    free(t->next);
    t->next = NULL;
    return head;
}

struct node *deleteValue(struct node *head, int value)
{
    struct node *t = head, *prev = NULL;
    while (t != NULL && t->data != value) { prev = t; t = t->next; }
    if (t == NULL) { printf("Value %d not found\n", value); return head; }
    if (prev == NULL) head = t->next;
    else prev->next = t->next;
    printf("Deleted value %d\n", value);
    free(t);
    return head;
}

void display(struct node *head)
{
    printf("List: ");
    for (struct node *t = head; t != NULL; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

int main(void)
{
    struct node *head = NULL;

    head = insertEnd(head, 20);   display(head);
    head = insertEnd(head, 30);   display(head);
    head = insertBegin(head, 10); display(head);
    head = insertEnd(head, 50);   display(head);
    head = insertPos(head, 40, 4); display(head);
    head = insertPos(head, 99, 9); display(head);

    head = deleteBegin(head);     display(head);
    head = deleteEnd(head);       display(head);
    head = deleteValue(head, 30); display(head);
    head = deleteValue(head, 77); display(head);

    while (head != NULL) head = deleteBegin(head);
```

```
    display(head);
    return 0;
}
```

Output

```
List: 20 -> NULL
List: 20 -> 30 -> NULL
List: 10 -> 20 -> 30 -> NULL
List: 10 -> 20 -> 30 -> 50 -> NULL
List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Invalid position 9
List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Deleted 10 from beginning
List: 20 -> 30 -> 40 -> 50 -> NULL
Deleted 50 from end
List: 20 -> 30 -> 40 -> NULL
Deleted value 30
List: 20 -> 40 -> NULL
Value 77 not found
List: 20 -> 40 -> NULL
Deleted 20 from beginning
Deleted 40 from beginning
List: NULL
```

Result: The program was executed successfully and the output was verified.

Program 2(ii) Reverse a linked list - iteratively and recursively

Aim

To write a program to reverse a singly linked list using iteration and recursion.

Algorithm

1. Iterative: use three pointers prev, curr and next. For every node: next = curr->next; curr->next = prev; prev = curr; curr = next. At the end prev is the new head.
2. Recursive: if the list is empty or has one node return it. Otherwise reverse the rest (head->next), then set head->next->next = head and head->next = NULL.
3. Both methods take O(n) time; recursion uses O(n) stack space.

Program

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *push(struct node *head, int value)   /* insert at end */
{
    struct node *n = malloc(sizeof(struct node)), *t = head;
    n->data = value;
    n->next = NULL;
    if (head == NULL) return n;
```

```
    while (t->next) t = t->next;
    t->next = n;
    return head;
}

void display(const char *msg, struct node *head)
{
    printf("%s", msg);
    for (struct node *t = head; t; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

struct node *reverseIterative(struct node *head)
{
    struct node *prev = NULL, *curr = head, *next;
    while (curr != NULL) {
        next = curr->next;      /* save next node   */
        curr->next = prev;      /* reverse the link */
        prev = curr;            /* move prev ahead  */
        curr = next;            /* move curr ahead  */
    }
    return prev;
}

struct node *reverseRecursive(struct node *head)
{
    if (head == NULL || head->next == NULL)
        return head;
    struct node *rest = reverseRecursive(head->next);
    head->next->next = head;
    head->next = NULL;
    return rest;
}

int main(void)
{
    struct node *head = NULL;
    for (int i = 1; i <= 6; i++)
        head = push(head, i * 10);

    display("Original list           : ", head);
    head = reverseIterative(head);
    display("Reversed (iterative)    : ", head);
    head = reverseRecursive(head);
    display("Reversed again (recursive): ", head);

    while (head) { struct node *t = head; head = head->next; free(t); }
    return 0;
}
```

Output

```
Original list           : 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL
Reversed (iterative)    : 60 -> 50 -> 40 -> 30 -> 20 -> 10 -> NULL
Reversed again (recursive): 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL
```

Result: The program was executed successfully and the output was verified.

Program 2(iii) Problems on linked list traversal and manipulation

Aim

To solve common problems on linked lists: count and sum of nodes, middle node, n-th node from the end, cycle detection and merging two sorted lists.

Algorithm

1. Length / sum: traverse once from head to NULL.
2. Middle node: slow pointer moves 1 step, fast pointer moves 2 steps; when fast reaches the end, slow is at the middle.
3. N-th from end: move the first pointer n steps ahead, then move both until the first reaches NULL.
4. Cycle detection (Floyd): if slow and fast pointers ever meet, the list has a cycle.
5. Merge two sorted lists: repeatedly attach the smaller front node to the result.

Program

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *build(int a[], int n)
{
    struct node *head = NULL, *tail = NULL;
    for (int i = 0; i < n; i++) {
        struct node *x = malloc(sizeof(struct node));
        x->data = a[i];
        x->next = NULL;
        if (head == NULL) head = tail = x;
        else { tail->next = x; tail = x; }
    }
    return head;
}

void display(const char *msg, struct node *head)
{
    printf("%s", msg);
    for (struct node *t = head; t; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

int length(struct node *head)
{
    int c = 0;
    for (; head; head = head->next) c++;
    return c;
}

int sum(struct node *head)
{
    int s = 0;
    for (; head; head = head->next) s += head->data;
    return s;
}

struct node *middle(struct node *head)
```

```
{
    struct node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

struct node *nthFromEnd(struct node *head, int n)
{
    struct node *first = head, *second = head;
    for (int i = 0; i < n; i++) {
        if (first == NULL) return NULL;
        first = first->next;
    }
    while (first) { first = first->next; second = second->next; }
    return second;
}

int hasCycle(struct node *head)
{
    struct node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1;
    }
    return 0;
}

struct node *mergeSorted(struct node *a, struct node *b)
{
    struct node dummy, *tail = &dummy;
    dummy.next = NULL;
    while (a && b) {
        if (a->data <= b->data) { tail->next = a; a = a->next; }
        else                    { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}

int main(void)
{
    int x[] = {5, 10, 15, 20, 25, 30, 35};
    int p[] = {1, 4, 9, 12};
    int q[] = {2, 3, 10, 20, 40};
    struct node *list = build(x, 7);

    display("List            : ", list);
    printf("Length          : %d\n", length(list));
    printf("Sum of nodes    : %d\n", sum(list));
    printf("Middle node     : %d\n", middle(list)->data);
    printf("3rd from end    : %d\n", nthFromEnd(list, 3)->data);

    printf("Cycle present?  : %s\n", hasCycle(list) ? "Yes" : "No");
    struct node *last = list;
    while (last->next) last = last->next;
    last->next = list->next->next;          /* create a cycle */
    printf("After linking tail to 3rd node, cycle present? : %s\n",
           hasCycle(list) ? "Yes" : "No");
    last->next = NULL;                      /* remove the cycle */
```

```
    struct node *a = build(p, 4), *b = build(q, 5);
    display("\nSorted list A   : ", a);
    display("Sorted list B   : ", b);
    display("Merged list     : ", mergeSorted(a, b));
    return 0;
}
```

Output

```
List            : 5 -> 10 -> 15 -> 20 -> 25 -> 30 -> 35 -> NULL
Length          : 7
Sum of nodes    : 140
Middle node     : 20
3rd from end    : 25
Cycle present?  : No
After linking tail to 3rd node, cycle present? : Yes

Sorted list A   : 1 -> 4 -> 9 -> 12 -> NULL
Sorted list B   : 2 -> 3 -> 10 -> 20 -> 40 -> NULL
Merged list     : 1 -> 2 -> 3 -> 4 -> 9 -> 10 -> 12 -> 20 -> 40 -> NULL
```

Result: The program was executed successfully and the output was verified.

Exercise 3: Linked List Applications and Deque

Program 3(i) Detect and remove duplicates from a linked list

Aim

To write a program to detect and remove duplicate elements from a linked list.

Algorithm

1. Take each node as the reference node, starting from the head.
2. Scan all following nodes; if a node has the same data as the reference node, unlink it and free it.
3. Move the reference to its next node and repeat. Time O(n^2), no extra memory.

Program

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *build(int a[], int n)
{
    struct node *head = NULL, *tail = NULL;
    for (int i = 0; i < n; i++) {
        struct node *x = malloc(sizeof(struct node));
        x->data = a[i];
        x->next = NULL;
        if (head == NULL) head = tail = x;
        else { tail->next = x; tail = x; }
    }
    return head;
}

void display(const char *msg, struct node *head)
{
    printf("%s", msg);
    for (struct node *t = head; t; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

int removeDuplicates(struct node *head)
{
    int removed = 0;
    for (struct node *ref = head; ref != NULL; ref = ref->next) {
        struct node *prev = ref, *cur = ref->next;
        while (cur != NULL) {
            if (cur->data == ref->data) {
                printf("  duplicate %d found and removed\n", cur->data);
                prev->next = cur->next;
                free(cur);
                cur = prev->next;
                removed++;
            } else {
                prev = cur;
                cur = cur->next;
            }
        }
```

```
    }
    return removed;
}

int main(void)
{
    int a[] = {10, 20, 10, 30, 20, 40, 10, 50, 30};
    struct node *head = build(a, 9);

    display("Original list : ", head);
    int r = removeDuplicates(head);
    display("After removal : ", head);
    printf("Total duplicates removed = %d\n", r);
    return 0;
}
```

Output

```
Original list : 10 -> 20 -> 10 -> 30 -> 20 -> 40 -> 10 -> 50 -> 30 -> NULL
  duplicate 10 found and removed
  duplicate 10 found and removed
  duplicate 20 found and removed
  duplicate 30 found and removed
After removal : 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Total duplicates removed = 4
```

Result: The program was executed successfully and the output was verified.

Program 3(ii) Polynomial representation and addition using linked lists

Aim

To represent polynomials using linked lists and perform polynomial addition.

Algorithm

1. Each node stores a coefficient, an exponent and a next pointer; terms are kept in decreasing order of exponent.
2. Compare the exponents of the current terms of P1 and P2.
3. If exponents are equal, add the coefficients (skip the term if the sum is 0); otherwise copy the term with the larger exponent.
4. Copy the remaining terms of whichever polynomial is left.

Program

```
#include <stdio.h>
#include <stdlib.h>

struct term {
    int coef, exp;
    struct term *next;
};

struct term *append(struct term *head, int coef, int exp)
{
    struct term *n = malloc(sizeof(struct term)), *t = head;
    n->coef = coef;
    n->exp = exp;
```

```
    n->next = NULL;
    if (head == NULL) return n;
    while (t->next) t = t->next;
    t->next = n;
    return head;
}

void printPoly(const char *name, struct term *p)
{
    printf("%s = ", name);
    if (p == NULL) { printf("0\n"); return; }
    for (struct term *t = p; t; t = t->next) {
        int c = t->coef;
        if (t != p) printf(c < 0 ? " - " : " + ");
        else if (c < 0) printf("-");
        if (c < 0) c = -c;
        if (c != 1 || t->exp == 0) printf("%d", c);
        if (t->exp > 1) printf("x^%d", t->exp);
        else if (t->exp == 1) printf("x");
    }
    printf("\n");
}

struct term *addPoly(struct term *a, struct term *b)
{
    struct term *r = NULL;
    while (a && b) {
        if (a->exp == b->exp) {
            int s = a->coef + b->coef;
            if (s != 0) r = append(r, s, a->exp);
            a = a->next;
            b = b->next;
        } else if (a->exp > b->exp) {
            r = append(r, a->coef, a->exp);
            a = a->next;
        } else {
            r = append(r, b->coef, b->exp);
            b = b->next;
        }
    }
    for (; a; a = a->next) r = append(r, a->coef, a->exp);
    for (; b; b = b->next) r = append(r, b->coef, b->exp);
    return r;
}

int main(void)
{
    struct term *p1 = NULL, *p2 = NULL, *p3 = NULL, *sum;

    p1 = append(p1, 5, 3); p1 = append(p1, 4, 2);
    p1 = append(p1, 2, 1); p1 = append(p1, 7, 0);

    p2 = append(p2, 3, 3); p2 = append(p2, -4, 2);
    p2 = append(p2, 6, 0);

    printPoly("P1(x)", p1);
    printPoly("P2(x)", p2);
    sum = addPoly(p1, p2);
    printPoly("P1 + P2", sum);

    p3 = append(p3, 1, 4); p3 = append(p3, -1, 1);
    printf("\n");
    printPoly("P3(x)", p3);
    printPoly("P1 + P3", addPoly(p1, p3));
    return 0;
```

```
}
```

Output

```
P1(x) = 5x^3 + 4x^2 + 2x + 7
P2(x) = 3x^3 - 4x^2 + 6
P1 + P2 = 8x^3 + 2x + 13

P3(x) = x^4 - x
P1 + P3 = x^4 + 5x^3 + 4x^2 + x + 7
```

Result: The program was executed successfully and the output was verified.

Program 3(iii) Double-ended queue (deque) with essential operations

Aim

To implement a double-ended queue (deque) using a circular array with insertion and deletion at both ends.

Algorithm

1. Keep front (index of the first element) and count (number of elements); the rear position is (front + count - 1) % SIZE.
2. Insert at front: front = (front - 1 + SIZE) % SIZE, store the value. Insert at rear: store at (front + count) % SIZE.
3. Delete from front: read a[front], then front = (front + 1) % SIZE. Delete from rear: read the element at the rear position.
4. Check isFull (count == SIZE) before inserting and isEmpty (count == 0) before deleting.

Program

```c
#include <stdio.h>
#define SIZE 5

struct deque {
    int a[SIZE];
    int front, count;
};

void init(struct deque *d) { d->front = 0; d->count = 0; }
int isEmpty(struct deque *d) { return d->count == 0; }
int isFull(struct deque *d)  { return d->count == SIZE; }

void insertFront(struct deque *d, int x)
{
    if (isFull(d)) { printf("Deque overflow, cannot insert %d at front\n", x); return; }
    d->front = (d->front - 1 + SIZE) % SIZE;
    d->a[d->front] = x;
    d->count++;
    printf("Inserted %d at front\n", x);
}

void insertRear(struct deque *d, int x)
{
    if (isFull(d)) { printf("Deque overflow, cannot insert %d at rear\n", x); return; }
    d->a[(d->front + d->count) % SIZE] = x;
    d->count++;
```

```
    printf("Inserted %d at rear\n", x);
}

void deleteFront(struct deque *d)
{
    if (isEmpty(d)) { printf("Deque underflow, nothing to delete at front\n"); return; }
    printf("Deleted %d from front\n", d->a[d->front]);
    d->front = (d->front + 1) % SIZE;
    d->count--;
}

void deleteRear(struct deque *d)
{
    if (isEmpty(d)) { printf("Deque underflow, nothing to delete at rear\n"); return; }
    printf("Deleted %d from rear\n", d->a[(d->front + d->count - 1) % SIZE]);
    d->count--;
}

void display(struct deque *d)
{
    printf("Deque (front -> rear): ");
    if (isEmpty(d)) printf("empty");
    for (int i = 0; i < d->count; i++)
        printf("%d ", d->a[(d->front + i) % SIZE]);
    printf("\n");
}

int main(void)
{
    struct deque d;
    init(&d);

    insertRear(&d, 10);   insertRear(&d, 20);
    insertFront(&d, 5);   insertFront(&d, 1);
    insertRear(&d, 30);   display(&d);
    insertRear(&d, 40);                          /* overflow */

    printf("Front element = %d, Rear element = %d\n",
           d.a[d.front], d.a[(d.front + d.count - 1) % SIZE]);

    deleteFront(&d);      display(&d);
    deleteRear(&d);       display(&d);
    insertFront(&d, 99);  display(&d);

    while (!isEmpty(&d)) deleteFront(&d);
    display(&d);
    deleteRear(&d);                              /* underflow */
    return 0;
}
```

Output

```
Inserted 10 at rear
Inserted 20 at rear
Inserted 5 at front
Inserted 1 at front
Inserted 30 at rear
Deque (front -> rear): 1 5 10 20 30
Deque overflow, cannot insert 40 at rear
Front element = 1, Rear element = 30
Deleted 1 from front
Deque (front -> rear): 5 10 20 30
Deleted 30 from rear
```

```
Deque (front -> rear): 5 10 20
Inserted 99 at front
Deque (front -> rear): 99 5 10 20
Deleted 99 from front
Deleted 5 from front
Deleted 10 from front
Deleted 20 from front
Deque (front -> rear): empty
Deque underflow, nothing to delete at rear
```

Result: The program was executed successfully and the output was verified.

Exercise 4: Stack ADT and Applications

Program 4(i) Stack ADT using arrays and linked lists

Aim

To implement the Stack ADT (push, pop, peek, display) using an array and using a linked list.

Algorithm

1. Stack follows LIFO: the last element pushed is the first one popped.
2. Array stack: keep an index top (-1 when empty). push: top++ then store; pop: read then top--. Overflow when top == SIZE-1.
3. Linked stack: push inserts a node at the head; pop removes the head node. No fixed size limit.
4. Underflow is reported when pop/peek is attempted on an empty stack.

Program

```c
#include <stdio.h>
#include <stdlib.h>
#define SIZE 4

/* ---------- Stack using array ---------- */
int stack[SIZE], top = -1;

void push(int x)
{
    if (top == SIZE - 1) { printf("  Stack overflow, cannot push %d\n", x); return; }
    stack[++top] = x;
    printf("  Pushed %d\n", x);
}

void pop(void)
{
    if (top == -1) { printf("  Stack underflow\n"); return; }
    printf("  Popped %d\n", stack[top--]);
}

void display(void)
{
    printf("  Stack (top to bottom): ");
    if (top == -1) printf("empty");
    for (int i = top; i >= 0; i--) printf("%d ", stack[i]);
    printf("\n");
}

/* ---------- Stack using linked list ---------- */
struct node { int data; struct node *next; };
struct node *head = NULL;

void lpush(int x)
{
    struct node *n = malloc(sizeof(struct node));
    n->data = x;
    n->next = head;
    head = n;
    printf("  Pushed %d\n", x);
}

void lpop(void)
{
```

```
    if (head == NULL) { printf("  Stack underflow\n"); return; }
    struct node *t = head;
    printf("  Popped %d\n", t->data);
    head = head->next;
    free(t);
}

void ldisplay(void)
{
    printf("  Stack (top to bottom): ");
    if (head == NULL) printf("empty");
    for (struct node *t = head; t; t = t->next) printf("%d ", t->data);
    printf("\n");
}

int main(void)
{
    printf("A. Stack using array (size %d)\n", SIZE);
    push(10); push(20); push(30); push(40); push(50);
    display();
    printf("  Peek = %d\n", stack[top]);
    pop(); pop();
    display();

    printf("\nB. Stack using linked list\n");
    lpush(5); lpush(15); lpush(25);
    ldisplay();
    printf("  Peek = %d\n", head->data);
    lpop(); lpop(); lpop(); lpop();
    ldisplay();
    return 0;
}
```

Output

```
A. Stack using array (size 4)
  Pushed 10
  Pushed 20
  Pushed 30
  Pushed 40
  Stack overflow, cannot push 50
  Stack (top to bottom): 40 30 20 10
  Peek = 40
  Popped 40
  Popped 30
  Stack (top to bottom): 20 10

B. Stack using linked list
  Pushed 5
  Pushed 15
  Pushed 25
  Stack (top to bottom): 25 15 5
  Peek = 25
  Popped 25
  Popped 15
  Popped 5
  Stack underflow
  Stack (top to bottom): empty
```

Result: The program was executed successfully and the output was verified.

Program 4(ii) Infix to postfix conversion and postfix evaluation

Aim

To convert an infix expression to postfix using a stack and evaluate the postfix expression.

Algorithm

1. Scan the infix expression left to right. Operands go directly to the output.
2. "(" is pushed; on ")" pop to output until "(" is found.
3. For an operator, pop operators of higher or equal precedence from the stack to the output, then push the operator (^ is right associative).
4. At the end pop all remaining operators.
5. Evaluation: scan postfix; push operands; on an operator pop two values (b then a), compute a op b and push the result. The final stack value is the answer.

Program

```c
#include <stdio.h>
#include <ctype.h>
#include <math.h>
#include <string.h>

char stk[50];
int top = -1;

void push(char c) { stk[++top] = c; }
char pop(void)    { return stk[top--]; }
char peek(void)   { return stk[top]; }

int prec(char c)
{
    switch (c) {
        case '^': return 3;
        case '*': case '/': return 2;
        case '+': case '-': return 1;
    }
    return 0;
}

void infixToPostfix(const char *in, char *out)
{
    int k = 0;
    top = -1;
    for (int i = 0; in[i]; i++) {
        char c = in[i];
        if (isalnum(c))
            out[k++] = c;
        else if (c == '(')
            push(c);
        else if (c == ')') {
            while (top != -1 && peek() != '(') out[k++] = pop();
            pop();                                      /* discard '(' */
        } else {
            while (top != -1 && peek() != '(' &&
                   (prec(peek()) > prec(c) ||
                    (prec(peek()) == prec(c) && c != '^')))
                out[k++] = pop();
            push(c);
        }
    }
```

```
    while (top != -1) out[k++] = pop();
    out[k] = '\0';
}

int evalPostfix(const char *p)
{
    int s[50], t = -1;
    for (int i = 0; p[i]; i++) {
        char c = p[i];
        if (isdigit(c))
            s[++t] = c - '0';
        else {
            int b = s[t--], a = s[t--];
            switch (c) {
                case '+': s[++t] = a + b; break;
                case '-': s[++t] = a - b; break;
                case '*': s[++t] = a * b; break;
                case '/': s[++t] = a / b; break;
                case '^': s[++t] = (int)pow(a, b); break;
            }
        }
    }
    return s[t];
}

int main(void)
{
    const char *exprs[] = { "A+B*C", "(A+B)*C-D/E", "A^B^C", "A*(B+C)/D" };
    const char *nums[]  = { "2+3*4", "(2+3)*4-6/2", "2^3^2", "9*(4+2)/3" };
    char post[50];

    printf("Infix to postfix conversion\n");
    for (int i = 0; i < 4; i++) {
        infixToPostfix(exprs[i], post);
        printf("  %-14s ->  %s\n", exprs[i], post);
    }

    printf("\nPostfix evaluation\n");
    for (int i = 0; i < 4; i++) {
        infixToPostfix(nums[i], post);
        printf("  %-14s ->  %-10s =  %d\n", nums[i], post, evalPostfix(post));
    }
    return 0;
}
```

Output

```
Infix to postfix conversion
  A+B*C          ->  ABC*+
  (A+B)*C-D/E    ->  AB+C*DE/-
  A^B^C          ->  ABC^^
  A*(B+C)/D      ->  ABC+*D/

Postfix evaluation
  2+3*4          ->  234*+      =  14
  (2+3)*4-6/2    ->  23+4*62/-  =  17
  2^3^2          ->  232^^      =  512
  9*(4+2)/3      ->  942+*3/    =  18
```

Result: The program was executed successfully and the output was verified.

Program 4(iii) Parentheses matching and palindrome check using a stack

Aim

To check balanced parentheses in an expression and to check whether a string is a palindrome using a stack.

Algorithm

1. Balanced parentheses: push every opening bracket ([{; on a closing bracket pop and check that it is the matching opening bracket.
2. Expression is balanced only if there is no mismatch and the stack is empty at the end.
3. Palindrome: push all characters on the stack, then pop them one by one and compare with the string from the beginning (reverse order comparison).
4. If every character matches, the string is a palindrome.

Program

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char stk[100];
int top = -1;

int balanced(const char *e)
{
    top = -1;
    for (int i = 0; e[i]; i++) {
        char c = e[i];
        if (c == '(' || c == '[' || c == '{')
            stk[++top] = c;
        else if (c == ')' || c == ']' || c == '}') {
            if (top == -1) return 0;
            char o = stk[top--];
            if ((c == ')' && o != '(') ||
                (c == ']' && o != '[') ||
                (c == '}' && o != '{'))
                return 0;
        }
    }
    return top == -1;
}

int palindrome(const char *s)
{
    char clean[100];
    int n = 0;
    for (int i = 0; s[i]; i++)
        if (isalnum((unsigned char)s[i]))
            clean[n++] = tolower((unsigned char)s[i]);
    clean[n] = '\0';

    top = -1;
    for (int i = 0; i < n; i++) stk[++top] = clean[i];
    for (int i = 0; i < n; i++)
        if (clean[i] != stk[top--]) return 0;
    return 1;
}

int main(void)
{
```

```c
    const char *e[] = { "{a+(b*c)-[d/e]}", "((a+b)*c", "[(a+b)}", "a+b)" };
    const char *w[] = { "madam", "level", "Was it a car or a cat I saw", "stack" };

    printf("Parentheses matching\n");
    for (int i = 0; i < 4; i++)
        printf("  %-18s : %s\n", e[i], balanced(e[i]) ? "Balanced" : "Not balanced");

    printf("\nPalindrome check\n");
    for (int i = 0; i < 4; i++)
        printf("  \"%s\" : %s\n", w[i], palindrome(w[i]) ? "Palindrome" : "Not a palindrome");
    return 0;
}
```

Output

```
Parentheses matching
  {a+(b*c)-[d/e]}    : Balanced
  ((a+b)*c           : Not balanced
  [(a+b)}            : Not balanced
  a+b)               : Not balanced

Palindrome check
  "madam" : Palindrome
  "level" : Palindrome
  "Was it a car or a cat I saw" : Palindrome
  "stack" : Not a palindrome
```

Result: The program was executed successfully and the output was verified.

Exercise 5: Queue ADT and Applications

Program 5(i) Linear queue using arrays and linked lists

Aim

To implement the Queue ADT (enqueue, dequeue, display) using an array and using a linked list.

Algorithm

1. Queue follows FIFO: insertion at the rear, deletion at the front.
2. Array queue: enqueue stores at a[++rear] (overflow if rear == SIZE-1); dequeue reads a[front++] (underflow if front > rear).
3. Linked queue: keep front and rear pointers; enqueue adds a node after rear, dequeue removes the front node.
4. In a linear array queue, freed front slots are not reused (this is solved by the circular queue).

Program

```c
#include <stdio.h>
#include <stdlib.h>
#define SIZE 4

/* ---------- Queue using array ---------- */
int q[SIZE], front = 0, rear = -1;

void enqueue(int x)
{
    if (rear == SIZE - 1) { printf("  Queue overflow, cannot insert %d\n", x); return; }
    q[++rear] = x;
    printf("  Enqueued %d\n", x);
}

void dequeue(void)
{
    if (front > rear) { printf("  Queue underflow\n"); return; }
    printf("  Dequeued %d\n", q[front++]);
}

void display(void)
{
    printf("  Queue (front to rear): ");
    if (front > rear) printf("empty");
    for (int i = front; i <= rear; i++) printf("%d ", q[i]);
    printf("\n");
}

/* ---------- Queue using linked list ---------- */
struct node { int data; struct node *next; };
struct node *f = NULL, *r = NULL;

void lenqueue(int x)
{
    struct node *n = malloc(sizeof(struct node));
    n->data = x;
    n->next = NULL;
    if (r == NULL) f = r = n;
    else { r->next = n; r = n; }
    printf("  Enqueued %d\n", x);
}
```

```c
void ldequeue(void)
{
    if (f == NULL) { printf("  Queue underflow\n"); return; }
    struct node *t = f;
    printf("  Dequeued %d\n", t->data);
    f = f->next;
    if (f == NULL) r = NULL;
    free(t);
}

void ldisplay(void)
{
    printf("  Queue (front to rear): ");
    if (f == NULL) printf("empty");
    for (struct node *t = f; t; t = t->next) printf("%d ", t->data);
    printf("\n");
}

int main(void)
{
    printf("A. Queue using array (size %d)\n", SIZE);
    enqueue(10); enqueue(20); enqueue(30); enqueue(40); enqueue(50);
    display();
    dequeue(); dequeue();
    display();
    enqueue(60);                    /* still overflow: freed slots not reused */

    printf("\nB. Queue using linked list\n");
    lenqueue(1); lenqueue(2); lenqueue(3);
    ldisplay();
    ldequeue(); ldequeue(); ldequeue(); ldequeue();
    ldisplay();
    return 0;
}
```

Output

```
A. Queue using array (size 4)
  Enqueued 10
  Enqueued 20
  Enqueued 30
  Enqueued 40
  Queue overflow, cannot insert 50
  Queue (front to rear): 10 20 30 40
  Dequeued 10
  Dequeued 20
  Queue (front to rear): 30 40
  Queue overflow, cannot insert 60

B. Queue using linked list
  Enqueued 1
  Enqueued 2
  Enqueued 3
  Queue (front to rear): 1 2 3
  Dequeued 1
  Dequeued 2
  Dequeued 3
  Queue underflow
  Queue (front to rear): empty
```

Result: The program was executed successfully and the output was verified.

Program 5(ii) Circular queue using arrays

Aim

To implement a circular queue using an array so that freed positions are reused.

Algorithm

1. Use front, rear and count. Positions wrap around using the modulus operator.
2. Enqueue: if count == SIZE -> overflow; else rear = (rear + 1) % SIZE, store value, count++.
3. Dequeue: if count == 0 -> underflow; else read a[front], front = (front + 1) % SIZE, count--.
4. Display elements from front for count positions with wrap-around.

Program

```c
#include <stdio.h>
#define SIZE 5

int q[SIZE];
int front = 0, rear = -1, count = 0;

void enqueue(int x)
{
    if (count == SIZE) { printf("  Queue overflow, cannot insert %d\n", x); return; }
    rear = (rear + 1) % SIZE;
    q[rear] = x;
    count++;
    printf("  Enqueued %d (rear index = %d)\n", x, rear);
}

void dequeue(void)
{
    if (count == 0) { printf("  Queue underflow\n"); return; }
    printf("  Dequeued %d (front index was %d)\n", q[front], front);
    front = (front + 1) % SIZE;
    count--;
}

void display(void)
{
    printf("  Queue: ");
    if (count == 0) printf("empty");
    for (int i = 0, k = front; i < count; i++, k = (k + 1) % SIZE)
        printf("%d ", q[k]);
    printf("  [front=%d, rear=%d, count=%d]\n", front, rear, count);
}

int main(void)
{
    enqueue(11); enqueue(22); enqueue(33); enqueue(44); enqueue(55);
    display();
    enqueue(66);                        /* full */
    dequeue(); dequeue();
    display();
    enqueue(66); enqueue(77);           /* reuses freed slots */
    display();
    while (count > 0) dequeue();
    display();
    dequeue();                          /* empty */
    return 0;
```

```
}
```

Output

```
  Enqueued 11 (rear index = 0)
  Enqueued 22 (rear index = 1)
  Enqueued 33 (rear index = 2)
  Enqueued 44 (rear index = 3)
  Enqueued 55 (rear index = 4)
  Queue: 11 22 33 44 55   [front=0, rear=4, count=5]
  Queue overflow, cannot insert 66
  Dequeued 11 (front index was 0)
  Dequeued 22 (front index was 1)
  Queue: 33 44 55   [front=2, rear=4, count=3]
  Enqueued 66 (rear index = 0)
  Enqueued 77 (rear index = 1)
  Queue: 33 44 55 66 77   [front=2, rear=1, count=5]
  Dequeued 33 (front index was 2)
  Dequeued 44 (front index was 3)
  Dequeued 55 (front index was 4)
  Dequeued 66 (front index was 0)
  Dequeued 77 (front index was 1)
  Queue: empty  [front=2, rear=1, count=0]
  Queue underflow
```

Result: The program was executed successfully and the output was verified.

Program 5(iii) Palindrome check using a queue and a stack; priority queue

Aim

To check a string for palindrome using a queue together with a stack, and to implement a simple priority queue.

Algorithm

1. Palindrome: insert each character into both a queue (FIFO) and a stack (LIFO).
2. Remove one character from each and compare; the queue gives the string forwards, the stack gives it backwards.
3. Priority queue: each element has a priority; insertion keeps the array sorted by priority (lower number = higher priority).
4. Deletion always removes the front (highest priority) element.

Program

```c
#include <stdio.h>
#include <string.h>

/* ---------- Palindrome using queue + stack ---------- */
int isPalindrome(const char *s)
{
    char queue[50], stack[50];
    int f = 0, r = -1, top = -1, n = strlen(s);

    for (int i = 0; i < n; i++) {
        queue[++r] = s[i];
```

```
        stack[++top] = s[i];
    }
    while (f <= r) {
        if (queue[f++] != stack[top--])
            return 0;
    }
    return 1;
}

/* ---------- Priority queue ---------- */
struct item { char name[20]; int pr; };
struct item pq[10];
int size = 0;

void insert(const char *name, int pr)
{
    int i = size - 1;
    while (i >= 0 && pq[i].pr > pr) {      /* shift lower priority items */
        pq[i + 1] = pq[i];
        i--;
    }
    strcpy(pq[i + 1].name, name);
    pq[i + 1].pr = pr;
    size++;
    printf("  Inserted %-10s priority %d\n", name, pr);
}

void removeFront(void)
{
    if (size == 0) { printf("  Priority queue empty\n"); return; }
    printf("  Served   %-10s priority %d\n", pq[0].name, pq[0].pr);
    for (int i = 1; i < size; i++) pq[i - 1] = pq[i];
    size--;
}

int main(void)
{
    const char *w[] = { "racecar", "radar", "queue", "noon" };

    printf("Palindrome check using queue and stack\n");
    for (int i = 0; i < 4; i++)
        printf("  %-8s : %s\n", w[i], isPalindrome(w[i]) ? "Palindrome" : "Not a palindrome");

    printf("\nPriority queue (1 = highest priority)\n");
    insert("Task-A", 3);
    insert("Task-B", 1);
    insert("Task-C", 2);
    insert("Task-D", 1);
    printf("  Serving order:\n");
    while (size > 0) removeFront();
    removeFront();
    return 0;
}
```

Output

```
Palindrome check using queue and stack
  racecar  : Palindrome
  radar    : Palindrome
  queue    : Not a palindrome
  noon     : Palindrome

Priority queue (1 = highest priority)
```

```
  Inserted Task-A     priority 3
  Inserted Task-B     priority 1
  Inserted Task-C     priority 2
  Inserted Task-D     priority 1
  Serving order:
  Served   Task-B     priority 1
  Served   Task-D     priority 1
  Served   Task-C     priority 2
  Served   Task-A     priority 3
  Priority queue empty
```

Result: The program was executed successfully and the output was verified.

Exercise 6: Binary Search Trees and Traversals

Program 6(i) Binary Search Tree - create, insert and search

Aim

To create a Binary Search Tree (BST) and perform insertion and searching of elements.

Algorithm

1. In a BST every key in the left subtree is smaller and every key in the right subtree is larger than the node key.
2. Insert: compare with the root; go left if smaller, right if larger, until a NULL position is found and place the new node there.
3. Search: same path as insertion; stop when the key matches or NULL is reached.
4. Average time O(log n); worst case O(n) for a skewed tree.

Program

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int key;
    struct node *left, *right;
};

struct node *newNode(int key)
{
    struct node *n = malloc(sizeof(struct node));
    n->key = key;
    n->left = n->right = NULL;
    return n;
}

struct node *insert(struct node *root, int key)
{
    if (root == NULL) return newNode(key);
    if (key < root->key)       root->left  = insert(root->left, key);
    else if (key > root->key)  root->right = insert(root->right, key);
    else printf("  Duplicate %d ignored\n", key);
    return root;
}

struct node *search(struct node *root, int key)
{
    while (root != NULL && root->key != key) {
        printf("  visiting %d\n", root->key);
        root = (key < root->key) ? root->left : root->right;
    }
    return root;
}

void inorder(struct node *root)
{
    if (root) { inorder(root->left); printf("%d ", root->key); inorder(root->right); }
}

int main(void)
{
    int keys[] = {50, 30, 70, 20, 40, 60, 80, 30};
```

```c
    struct node *root = NULL;

    printf("Inserting:");
    for (int i = 0; i < 8; i++) printf(" %d", keys[i]);
    printf("\n");
    for (int i = 0; i < 8; i++) root = insert(root, keys[i]);

    printf("Inorder traversal (sorted order): ");
    inorder(root);
    printf("\n");

    int find[] = {60, 25};
    for (int i = 0; i < 2; i++) {
        printf("\nSearching %d\n", find[i]);
        if (search(root, find[i])) printf("  %d is present in the BST\n", find[i]);
        else                       printf("  %d is not present in the BST\n", find[i]);
    }
    return 0;
}
```

Output

```
Inserting: 50 30 70 20 40 60 80 30
  Duplicate 30 ignored
Inorder traversal (sorted order): 20 30 40 50 60 70 80

Searching 60
  visiting 50
  visiting 70
  60 is present in the BST

Searching 25
  visiting 50
  visiting 30
  visiting 20
  25 is not present in the BST
```

Result: The program was executed successfully and the output was verified.

Program 6(ii) BST - deletion, minimum, maximum, height and count

Aim

To perform deletion of a node from a BST and find its minimum, maximum, height and number of nodes.

Algorithm

1. Delete a leaf: simply remove it. Delete a node with one child: replace the node by its child.
2. Delete a node with two children: replace its key with the inorder successor (smallest key of the right subtree) and delete that successor.
3. Minimum = leftmost node, maximum = rightmost node.
4. Height = 1 + max(height(left), height(right)); count = 1 + count(left) + count(right).

Program

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int key;
    struct node *left, *right;
};

struct node *insert(struct node *root, int key)
{
    if (root == NULL) {
        struct node *n = malloc(sizeof(struct node));
        n->key = key;
        n->left = n->right = NULL;
        return n;
    }
    if (key < root->key) root->left = insert(root->left, key);
    else                 root->right = insert(root->right, key);
    return root;
}

struct node *minNode(struct node *r) { while (r->left)  r = r->left;  return r; }
struct node *maxNode(struct node *r) { while (r->right) r = r->right; return r; }

struct node *deleteNode(struct node *root, int key)
{
    if (root == NULL) { printf("  %d not found\n", key); return NULL; }
    if (key < root->key)       root->left  = deleteNode(root->left, key);
    else if (key > root->key)  root->right = deleteNode(root->right, key);
    else {
        if (root->left == NULL) {
            struct node *t = root->right; free(root); return t;
        }
        if (root->right == NULL) {
            struct node *t = root->left; free(root); return t;
        }
        struct node *s = minNode(root->right);      /* inorder successor */
        root->key = s->key;
        root->right = deleteNode(root->right, s->key);
    }
    return root;
}

int height(struct node *r)
{
    if (r == NULL) return 0;
    int l = height(r->left), rr = height(r->right);
    return 1 + (l > rr ? l : rr);
}

int count(struct node *r) { return r ? 1 + count(r->left) + count(r->right) : 0; }

void inorder(struct node *r)
{
    if (r) { inorder(r->left); printf("%d ", r->key); inorder(r->right); }
}

void show(struct node *root)
{
    printf("  Inorder: ");
    inorder(root);
    printf("\n  Min=%d Max=%d Height=%d Nodes=%d\n",
           minNode(root)->key, maxNode(root)->key, height(root), count(root));
```

```c
}

int main(void)
{
    int keys[] = {50, 30, 70, 20, 40, 60, 80, 35, 65};
    struct node *root = NULL;
    for (int i = 0; i < 9; i++) root = insert(root, keys[i]);

    printf("Initial tree\n");           show(root);
    printf("\nDelete 20 (leaf)\n");     root = deleteNode(root, 20);  show(root);
    printf("\nDelete 30 (two children)\n"); root = deleteNode(root, 30); show(root);
    printf("\nDelete 70 (two children)\n"); root = deleteNode(root, 70); show(root);
    printf("\nDelete 99 (absent)\n");   root = deleteNode(root, 99);
    return 0;
}
```

Output

```
Initial tree
  Inorder: 20 30 35 40 50 60 65 70 80
  Min=20 Max=80 Height=4 Nodes=9

Delete 20 (leaf)
  Inorder: 30 35 40 50 60 65 70 80
  Min=30 Max=80 Height=4 Nodes=8

Delete 30 (two children)
  Inorder: 35 40 50 60 65 70 80
  Min=35 Max=80 Height=4 Nodes=7

Delete 70 (two children)
  Inorder: 35 40 50 60 65 80
  Min=35 Max=80 Height=4 Nodes=6

Delete 99 (absent)
  99 not found
```

Result: The program was executed successfully and the output was verified.

Program 6(iii) Tree traversals - inorder, preorder, postorder and level order

Aim

To implement the tree traversal techniques (inorder, preorder, postorder, level order) on a binary tree.

Algorithm

1. Inorder: Left, Root, Right. Preorder: Root, Left, Right. Postorder: Left, Right, Root.
2. Level order (breadth first): use a queue; visit the front node, then enqueue its left and right children.
3. Inorder of a BST gives the keys in ascending order.

Program

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int key;
    struct node *left, *right;
};

struct node *insert(struct node *root, int key)
{
    if (root == NULL) {
        struct node *n = malloc(sizeof(struct node));
        n->key = key;
        n->left = n->right = NULL;
        return n;
    }
    if (key < root->key) root->left = insert(root->left, key);
    else                 root->right = insert(root->right, key);
    return root;
}

void inorder(struct node *r)
{ if (r) { inorder(r->left);  printf("%d ", r->key); inorder(r->right); } }

void preorder(struct node *r)
{ if (r) { printf("%d ", r->key); preorder(r->left); preorder(r->right); } }

void postorder(struct node *r)
{ if (r) { postorder(r->left); postorder(r->right); printf("%d ", r->key); } }

void levelOrder(struct node *root)
{
    struct node *queue[50];
    int f = 0, r = 0;
    if (root) queue[r++] = root;
    while (f < r) {
        struct node *t = queue[f++];
        printf("%d ", t->key);
        if (t->left)  queue[r++] = t->left;
        if (t->right) queue[r++] = t->right;
    }
}

int main(void)
{
    int keys[] = {45, 25, 65, 15, 35, 55, 75, 10, 30};
    struct node *root = NULL;
    for (int i = 0; i < 9; i++) root = insert(root, keys[i]);

    printf("Tree built from: 45 25 65 15 35 55 75 10 30\n\n");
    printf("Inorder     : "); inorder(root);    printf("\n");
    printf("Preorder    : "); preorder(root);   printf("\n");
    printf("Postorder   : "); postorder(root);  printf("\n");
    printf("Level order : "); levelOrder(root); printf("\n");
    return 0;
}
```

Output

```
Tree built from: 45 25 65 15 35 55 75 10 30

Inorder     : 10 15 25 30 35 45 55 65 75
Preorder    : 45 25 15 10 35 30 65 55 75
Postorder   : 10 15 30 35 25 55 75 65 45
Level order : 45 25 65 15 35 55 75 10 30
```

Result: The program was executed successfully and the output was verified.

Exercise 7: Hashing: Chaining and Open Addressing

Program 7(i) Hash table with separate chaining

Aim

To implement a hash table that resolves collisions using separate chaining (linked lists).

Algorithm

1. Hash function: index = key % TABLE_SIZE.
2. Each table slot holds the head of a linked list; keys with the same index are chained in that list.
3. Insert: add a new node at the head of the chain. Search: traverse the chain of that index. Delete: unlink the node from the chain.
4. Average time O(1) when the load factor is small.

Program

```c
#include <stdio.h>
#include <stdlib.h>
#define M 7

struct node { int key; struct node *next; };
struct node *table[M];

int hash(int key) { return key % M; }

void insert(int key)
{
    int i = hash(key);
    struct node *n = malloc(sizeof(struct node));
    n->key = key;
    n->next = table[i];
    table[i] = n;
    printf("  Inserted %3d at index %d\n", key, i);
}

int search(int key)
{
    for (struct node *t = table[hash(key)]; t; t = t->next)
        if (t->key == key) return 1;
    return 0;
}

void delete(int key)
{
    int i = hash(key);
    struct node *t = table[i], *prev = NULL;
    while (t && t->key != key) { prev = t; t = t->next; }
    if (t == NULL) { printf("  %d not found, cannot delete\n", key); return; }
    if (prev) prev->next = t->next; else table[i] = t->next;
    free(t);
    printf("  Deleted %d\n", key);
}

void display(void)
{
    for (int i = 0; i < M; i++) {
        printf("  [%d] ", i);
```

```
        for (struct node *t = table[i]; t; t = t->next)
            printf("%d -> ", t->key);
        printf("NULL\n");
    }
}

int main(void)
{
    int keys[] = {15, 11, 27, 8, 12, 22, 29, 36};
    printf("Hash function: key %% %d\n", M);
    for (int i = 0; i < 8; i++) insert(keys[i]);

    printf("\nHash table:\n");
    display();

    printf("\nSearch 22 : %s\n", search(22) ? "Found" : "Not found");
    printf("Search 50 : %s\n", search(50) ? "Found" : "Not found");

    printf("\n");
    delete(29);
    delete(100);
    printf("\nHash table after deletion:\n");
    display();
    return 0;
}
```

Output

```
Hash function: key % 7
  Inserted  15 at index 1
  Inserted  11 at index 4
  Inserted  27 at index 6
  Inserted   8 at index 1
  Inserted  12 at index 5
  Inserted  22 at index 1
  Inserted  29 at index 1
  Inserted  36 at index 1

Hash table:
  [0] NULL
  [1] 36 -> 29 -> 22 -> 8 -> 15 -> NULL
  [2] NULL
  [3] NULL
  [4] 11 -> NULL
  [5] 12 -> NULL
  [6] 27 -> NULL

Search 22 : Found
Search 50 : Not found

  Deleted 29
  100 not found, cannot delete

Hash table after deletion:
  [0] NULL
  [1] 36 -> 22 -> 8 -> 15 -> NULL
  [2] NULL
  [3] NULL
  [4] 11 -> NULL
```

```
  [5] 12 -> NULL
  [6] 27 -> NULL
```

Result: The program was executed successfully and the output was verified.

Program 7(ii) Hash table with open addressing - linear probing

Aim

To implement a hash table using open addressing with linear probing for collision resolution.

Algorithm

1. Hash function: index = key % SIZE. If the slot is occupied, try (index+1) % SIZE, (index+2) % SIZE ... until an empty slot is found.
2. Search follows the same probe sequence and stops on a match or an empty slot.
3. Insert fails (table full) after SIZE probes.
4. Linear probing may cause primary clustering of keys.

Program

```c
#include <stdio.h>
#define SIZE 10
#define EMPTY -1

int table[SIZE];

int hash(int key) { return key % SIZE; }

void insert(int key)
{
    int i = hash(key), probes = 0;
    while (table[i] != EMPTY && probes < SIZE) {
        printf("    collision at index %d\n", i);
        i = (i + 1) % SIZE;
        probes++;
    }
    if (probes == SIZE) { printf("  Table full, cannot insert %d\n", key); return; }
    table[i] = key;
    printf("  Inserted %3d at index %d\n", key, i);
}

int search(int key)
{
    int i = hash(key);
    for (int probes = 0; probes < SIZE && table[i] != EMPTY; probes++) {
        if (table[i] == key) return i;
        i = (i + 1) % SIZE;
    }
    return -1;
}

void display(void)
{
    for (int i = 0; i < SIZE; i++) {
        if (table[i] == EMPTY) printf("  [%d] --\n", i);
        else                   printf("  [%d] %d\n", i, table[i]);
    }
}

int main(void)
```

```c
{
    int keys[] = {23, 43, 1, 87, 33, 54, 64, 7};
    for (int i = 0; i < SIZE; i++) table[i] = EMPTY;

    printf("Hash function: key %% %d (linear probing)\n", SIZE);
    for (int i = 0; i < 8; i++) insert(keys[i]);

    printf("\nHash table:\n");
    display();

    int find[] = {87, 64, 99};
    printf("\n");
    for (int i = 0; i < 3; i++) {
        int p = search(find[i]);
        if (p >= 0) printf("Search %d : found at index %d\n", find[i], p);
        else        printf("Search %d : not found\n", find[i]);
    }
    return 0;
}
```

Output

```
Hash function: key % 10 (linear probing)
  Inserted  23 at index 3
    collision at index 3
  Inserted  43 at index 4
  Inserted   1 at index 1
  Inserted  87 at index 7
    collision at index 3
    collision at index 4
  Inserted  33 at index 5
    collision at index 4
    collision at index 5
  Inserted  54 at index 6
    collision at index 4
    collision at index 5
    collision at index 6
    collision at index 7
  Inserted  64 at index 8
    collision at index 7
    collision at index 8
  Inserted   7 at index 9

Hash table:
  [0] --
  [1] 1
  [2] --
  [3] 23
  [4] 43
  [5] 33
  [6] 54
  [7] 87
  [8] 64
  [9] 7

Search 87 : found at index 7
Search 64 : found at index 8
Search 99 : not found
```

Result: The program was executed successfully and the output was verified.

Exercise 8: Advanced Hashing and Applications

Program 8(i) Hash table - quadratic probing and double hashing

Aim

To implement open addressing with quadratic probing and double hashing and compare the placement of keys.

Algorithm

1. Quadratic probing: index = (h(key) + i*i) % SIZE for i = 0, 1, 2 ...
2. Double hashing: index = (h1(key) + i * h2(key)) % SIZE where h1 = key % SIZE and h2 = 7 - (key % 7).
3. Both methods reduce the clustering seen in linear probing.
4. The same set of keys is inserted by both methods and the resulting tables are displayed.

Program

```
#include <stdio.h>
#define SIZE 11
#define EMPTY -1

int h1(int k) { return k % SIZE; }
int h2(int k) { return 7 - (k % 7); }

int insertQuadratic(int t[], int key)
{
    for (int i = 0; i < SIZE; i++) {
        int idx = (h1(key) + i * i) % SIZE;
        if (t[idx] == EMPTY) { t[idx] = key; return i; }
    }
    return -1;
}

int insertDouble(int t[], int key)
{
    for (int i = 0; i < SIZE; i++) {
        int idx = (h1(key) + i * h2(key)) % SIZE;
        if (t[idx] == EMPTY) { t[idx] = key; return i; }
    }
    return -1;
}

void show(const char *name, int t[])
{
    printf("%s\n  index : ", name);
    for (int i = 0; i < SIZE; i++) printf("%4d", i);
    printf("\n  key   : ");
    for (int i = 0; i < SIZE; i++) {
        if (t[i] == EMPTY) printf("   -");
        else               printf("%4d", t[i]);
    }
    printf("\n");
}

int main(void)
{
    int keys[] = {22, 33, 44, 55, 11, 66, 77};
    int n = sizeof(keys) / sizeof(keys[0]);
    int tq[SIZE], td[SIZE];
```

```
    for (int i = 0; i < SIZE; i++) tq[i] = td[i] = EMPTY;

    printf("Keys: 22 33 44 55 11 66 77   (table size %d)\n\n", SIZE);
    printf("Probes needed (0 = no collision)\n");
    printf("  key  quadratic  double\n");
    for (int i = 0; i < n; i++) {
        int a = insertQuadratic(tq, keys[i]);
        int b = insertDouble(td, keys[i]);
        printf("  %3d  %8d  %6d\n", keys[i], a, b);
    }
    printf("\n");
    show("Quadratic probing table", tq);
    printf("\n");
    show("Double hashing table", td);
    return 0;
}
```

Output

```
Keys: 22 33 44 55 11 66 77   (table size 11)

Probes needed (0 = no collision)
  key  quadratic  double
   22         0       0
   33         1       1
   44         2       1
   55         3       1
   11         4       1
   66         5       1
   77        -1       1

Quadratic probing table
  index :    0   1   2   3   4   5   6   7   8   9  10
  key   :   22  33   -  66  44  11   -   -   -  55   -

Double hashing table
  index :    0   1   2   3   4   5   6   7   8   9  10
  key   :   22  55  33  11  66  44   -  77   -   -   -
```

Result: The program was executed successfully and the output was verified.

Program 8(ii) Hashing applications - word frequency count and two-sum

Aim

To use a hash table to count word frequencies and to find a pair of numbers with a given sum.

Algorithm

1. String hash: h = (h * 31 + character) % SIZE. Words with the same hash are chained.
2. Word count: for each word look it up in the table; increase its count if present, otherwise insert it with count 1.
3. Two-sum: for each number x check whether (target - x) is already in the hash table; if yes the pair is found, else store x. Time O(n).

Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define SIZE 11

struct entry {
    char word[20];
    int count;
    struct entry *next;
};
struct entry *table[SIZE];

unsigned hash(const char *s)
{
    unsigned h = 0;
    while (*s) h = (h * 31 + *s++) % SIZE;
    return h;
}

void addWord(const char *w)
{
    unsigned i = hash(w);
    for (struct entry *e = table[i]; e; e = e->next)
        if (strcmp(e->word, w) == 0) { e->count++; return; }
    struct entry *n = malloc(sizeof(struct entry));
    strcpy(n->word, w);
    n->count = 1;
    n->next = table[i];
    table[i] = n;
}

/* ---- Two-sum using a direct-address hash of seen values ---- */
int seen[200];

int main(void)
{
    char text[] = "to be or not to be that is the question to be";
    char *w = strtok(text, " ");
    while (w) { addWord(w); w = strtok(NULL, " "); }

    printf("Text: to be or not to be that is the question to be\n\n");
    printf("Word frequency (hash index, word, count)\n");
    for (int i = 0; i < SIZE; i++)
        for (struct entry *e = table[i]; e; e = e->next)
            printf("  [%2d] %-9s %d\n", i, e->word, e->count);

    int a[] = {8, 3, 11, 5, 14, 2};
    int target = 16, found = 0;
    printf("\nTwo-sum: array {8 3 11 5 14 2}, target = %d\n", target);
    for (int i = 0; i < 6; i++) {
        int need = target - a[i];
        if (need >= 0 && seen[need]) {
            printf("  Pair found: %d + %d = %d\n", need, a[i], target);
            found = 1;
        }
        seen[a[i]] = 1;
    }
    if (!found) printf("  No pair found\n");
    return 0;
}
```

Output

```
Text: to be or not to be that is the question to be

Word frequency (hash index, word, count)
  [ 0] to        3
  [ 2] or        1
  [ 4] is        1
  [ 4] that      1
  [ 4] not       1
  [ 4] be        3
  [ 5] the       1
  [10] question  1

Two-sum: array {8 3 11 5 14 2}, target = 16
  Pair found: 11 + 5 = 16
  Pair found: 14 + 2 = 16
```

Result: The program was executed successfully and the output was verified.

Exercise 9: Cache Implementation

Program 9(i) LRU cache using a hash map and doubly linked list

Aim

To implement a Least Recently Used (LRU) cache with get and put operations.

Algorithm

1. A doubly linked list keeps entries ordered from most recently used (head) to least recently used (tail).
2. A lookup (hash map / table indexed by key) gives the node in O(1).
3. get(key): if present, move the node to the head and return its value; otherwise report a miss.
4. put(key,value): if present, update and move to head; else insert at head. If capacity is exceeded, remove the tail node (the LRU entry).

Program

```
#include <stdio.h>
#include <stdlib.h>
#define CAP 3
#define MAXKEY 100

struct node {
    int key, value;
    struct node *prev, *next;
};

struct node *head = NULL, *tail = NULL;
struct node *map[MAXKEY];            /* direct hash: key -> node */
int size = 0;

void unlink_node(struct node *n)
{
    if (n->prev) n->prev->next = n->next; else head = n->next;
    if (n->next) n->next->prev = n->prev; else tail = n->prev;
}

void addFront(struct node *n)
{
    n->prev = NULL;
    n->next = head;
    if (head) head->prev = n;
    head = n;
    if (tail == NULL) tail = n;
}

int get(int key)
{
    struct node *n = map[key];
    if (n == NULL) { printf("  get(%d)    -> MISS\n", key); return -1; }
    unlink_node(n);
    addFront(n);
    printf("  get(%d)    -> HIT, value = %d\n", key, n->value);
    return n->value;
}

void put(int key, int value)
{
    struct node *n = map[key];
```

```
    if (n) {
        n->value = value;
        unlink_node(n);
        addFront(n);
        printf("  put(%d,%d) -> updated\n", key, value);
        return;
    }
    if (size == CAP) {
        struct node *old = tail;
        printf("  put(%d,%d) -> cache full, evicting key %d\n", key, value, old->key);
        unlink_node(old);
        map[old->key] = NULL;
        free(old);
        size--;
    } else {
        printf("  put(%d,%d) -> inserted\n", key, value);
    }
    n = malloc(sizeof(struct node));
    n->key = key;
    n->value = value;
    map[key] = n;
    addFront(n);
    size++;
}

void show(void)
{
    printf("    cache (MRU -> LRU): ");
    for (struct node *t = head; t; t = t->next)
        printf("(%d:%d) ", t->key, t->value);
    printf("\n");
}

int main(void)
{
    printf("LRU cache, capacity = %d\n", CAP);
    put(1, 10);  show();
    put(2, 20);  show();
    put(3, 30);  show();
    get(1);      show();
    put(4, 40);  show();
    get(2);
    put(3, 33);  show();
    put(5, 50);  show();
    get(1);
    get(3);      show();
    return 0;
}
```

Output

```
LRU cache, capacity = 3
  put(1,10) -> inserted
    cache (MRU -> LRU): (1:10)
  put(2,20) -> inserted
    cache (MRU -> LRU): (2:20) (1:10)
  put(3,30) -> inserted
    cache (MRU -> LRU): (3:30) (2:20) (1:10)
  get(1)    -> HIT, value = 10
    cache (MRU -> LRU): (1:10) (3:30) (2:20)
  put(4,40) -> cache full, evicting key 2
    cache (MRU -> LRU): (4:40) (1:10) (3:30)
```

```
  get(2)    -> MISS
  put(3,33) -> updated
    cache (MRU -> LRU): (3:33) (4:40) (1:10)
  put(5,50) -> cache full, evicting key 1
    cache (MRU -> LRU): (5:50) (3:33) (4:40)
  get(1)    -> MISS
  get(3)    -> HIT, value = 33
    cache (MRU -> LRU): (3:33) (5:50) (4:40)
```

Result: The program was executed successfully and the output was verified.

Program 9(ii) Cache simulation - direct mapped cache with hit ratio

Aim

To simulate a direct-mapped cache and compute the number of hits, misses and the hit ratio for a sequence of memory block accesses.

Algorithm

1. The cache has N lines. A memory block b maps to line = b % N (direct mapping); the tag is b / N.
2. On access: if the line is valid and its tag matches -> HIT; otherwise -> MISS and the block is loaded into the line (replacing the old one).
3. Hit ratio = hits / total accesses.

Program

```c
#include <stdio.h>
#define LINES 4

struct line { int valid, tag; };

int main(void)
{
    struct line cache[LINES] = {{0, 0}};
    int access[] = {0, 4, 0, 8, 1, 5, 1, 0, 4, 2, 6, 2, 0};
    int n = sizeof(access) / sizeof(access[0]);
    int hits = 0, misses = 0;

    printf("Direct mapped cache with %d lines\n\n", LINES);
    printf(" Block  Line  Tag  Result\n");
    printf(" -----  ----  ---  ------\n");
    for (int i = 0; i < n; i++) {
        int b = access[i];
        int idx = b % LINES, tag = b / LINES;
        if (cache[idx].valid && cache[idx].tag == tag) {
            hits++;
            printf(" %5d  %4d  %3d  HIT\n", b, idx, tag);
        } else {
            misses++;
            cache[idx].valid = 1;
            cache[idx].tag = tag;
            printf(" %5d  %4d  %3d  MISS\n", b, idx, tag);
        }
    }
    printf("\nTotal accesses = %d\n", n);
    printf("Hits           = %d\n", hits);
    printf("Misses         = %d\n", misses);
    printf("Hit ratio      = %.2f%%\n", 100.0 * hits / n);
    return 0;
```

```
}
```

Output

```
Direct mapped cache with 4 lines

 Block  Line  Tag  Result
 -----  ----  ---  ------
     0     0    0  MISS
     4     0    1  MISS
     0     0    0  MISS
     8     0    2  MISS
     1     1    0  MISS
     5     1    1  MISS
     1     1    0  MISS
     0     0    0  MISS
     4     0    1  MISS
     2     2    0  MISS
     6     2    1  MISS
     2     2    0  MISS
     0     0    0  MISS

Total accesses = 13
Hits           = 0
Misses         = 13
Hit ratio      = 0.00%
```

Result: The program was executed successfully and the output was verified.

Text Books and Reference Books

- Reema Thareja, Data Structures Using C, 2nd Edition, Oxford University Press, 2014.
- Mark Allen Weiss, Data Structures and Algorithm Analysis in C, 2nd Edition, Pearson.
- E. Horowitz, S. Sahni, Fundamentals of Data Structures in C, 2nd Edition, Universities Press.
- Seymour Lipschutz, Data Structures with C, Schaum's Outlines, McGraw Hill.
- Yashavant Kanetkar, Let Us C, BPB Publications.

Note: sample data is hard-coded in each program so that the output is reproducible. All outputs shown are the real output captured by compiling and running the code.